



VIT[®]
BHOPAL



PROJECT ON:-

Mini Banking Transaction Logger

Submitted by	MIHIR SINGH
Registration Number	26BAI10870
Academic Year	2026-2027
Slot	B21+B22+B23+E14+E21+E22

Introduction:

This project is a simple console-based banking transaction logger. It allows a user to record deposits and withdrawals, check the current balance, view transaction history and delete any wrong entry. The main aim of making this project is to practice the basic Python concepts taught in the course such as lists, dictionaries, functions, loops and conditionals. The program is completely menu driven and easy to use.

Problem Statement:

People often need to keep a record of their daily money transactions. Writing everything on paper is not convenient and calculating the balance manually can lead to mistakes. There is a need for a simple program that can store deposits and withdrawals, show the current balance and give a clear summary of all transactions. This project tries to solve that problem using only the basic programming concepts learned in class.

Objectives:

- **To create a simple program that can add deposit and withdrawal entries**
- **To calculate and display the current balance correctly**
- **To show all transactions and also filter them by type**
- **To allow the user to delete any transaction if needed**
- **To practice using lists, dictionaries, functions and loops**

Functional Requirements:

The project has three main modules:

1. Transaction Management Module

- Add a new deposit
- Add a new withdrawal
- Delete a transaction using its ID

2. Calculation Module

- Calculate current balance
- Calculate total deposits
- Calculate total withdrawals

3. Reporting Module

- Display all transactions
- Show only deposits
- Show only withdrawals
- Show account summary

Non-Functional Requirements:

1. **Usability** – The menu is simple and easy to understand.
2. **Reliability** – The program checks for invalid amounts and shows proper messages.
3. **Maintainability** – The code is divided into different functions so it is easy to understand and change.
4. **Performance** – The program works quickly even with many transactions because it uses simple list operations.
5. **Error Handling** – Wrong inputs like negative amounts or invalid IDs are handled properly.

System Architecture:

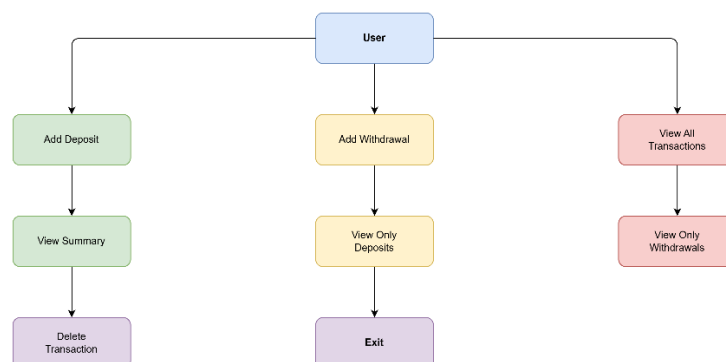
The program is divided into different parts:

- **main.py** controls the menu and overall flow
- **transactionmanager.py** handles adding and deleting transactions
- **calculator.py** does all the balance and total calculations
- **reporter.py** shows the transactions and summary
- **datahandler.py** helps in finding a transaction and creating new IDs.

Design Diagrams:

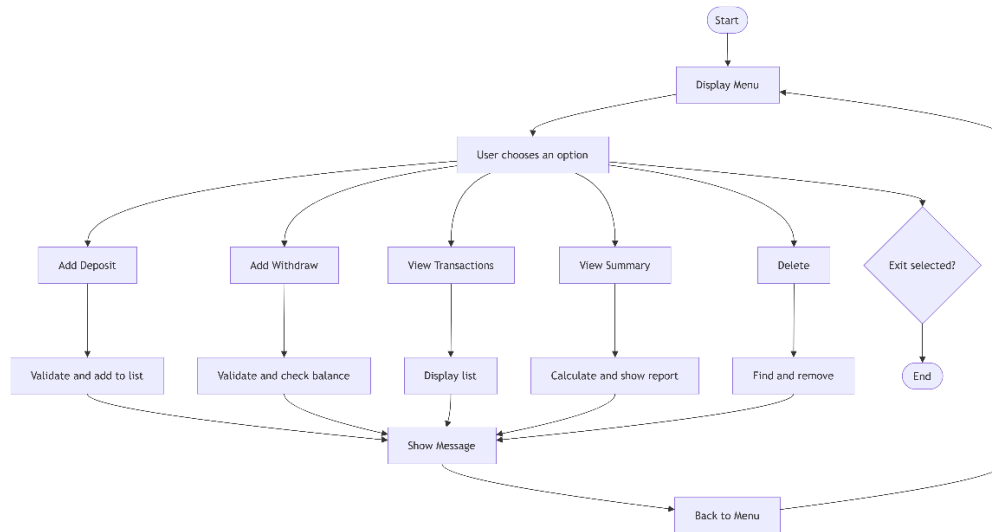
Use Case Diagram

The main actor is the User. The user can add deposit, add withdrawal, view transactions, view summary, view only deposits, view only withdrawals and delete a transaction.



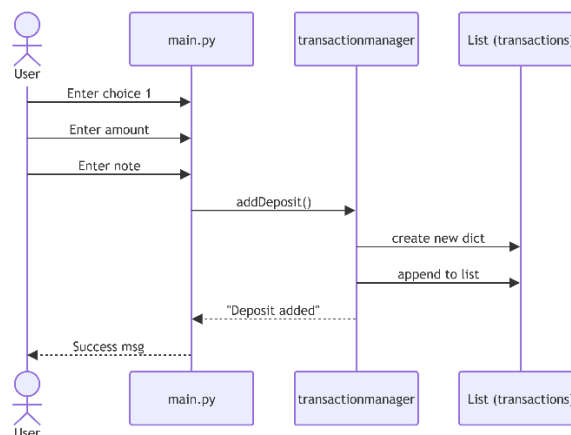
Workflow Diagram

The program starts and shows the menu. The user selects an option. Based on the choice the related function is called. After the operation is completed the menu is shown again until the user chooses to exit.



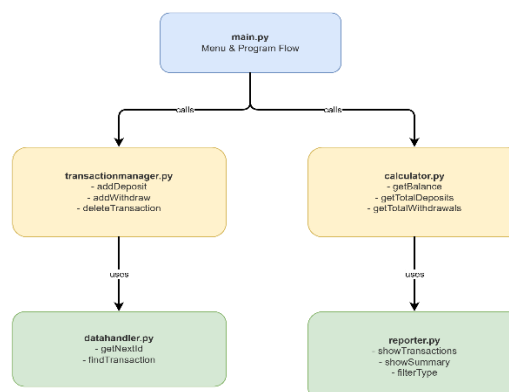
Sequence Diagram (for Add Deposit)

User enters the amount and note → main function calls addDeposit → new transaction is created and added to the list → success message is shown to the user.



Component Diagram

The main components are Main Controller, Transaction Manager, Calculator and Reporter. They work together to complete the tasks.



Design Decisions and Rationale:

- Console based program was chosen because it matches the level of the course and does not need any extra libraries.
- List of dictionaries is used to store transactions because it is easy to understand and taught in the syllabus.
- Separate functions are used for different tasks so that the code remains clean and modular.
- No file storage is used to keep the project simple and within the basic concepts.

Implementation Details:

The main data structure is a list. Each transaction is stored as a dictionary with keys like id, type, amount and note.

Functions are created for:

- Adding deposit and withdrawal
- Calculating balance and totals
- Displaying transactions
- Filtering by type
- Deleting a transaction

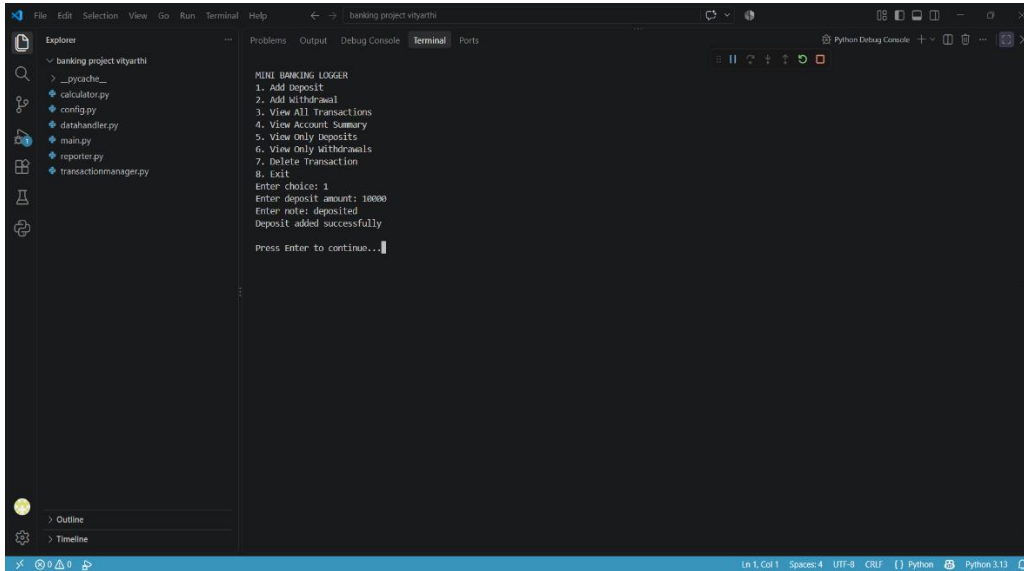
Testing Approach:

Testing was done manually by performing the following steps:

- Added some deposit and withdrawal entries
- Checked if the balance is calculated correctly
- Viewed only deposits and only withdrawals
- Deleted a transaction and confirmed it was removed
- Entered wrong values like negative amount and invalid ID to check error messages
- Checked the summary to confirm totals are correct

Screenshots / Results:

1.Add Deposit

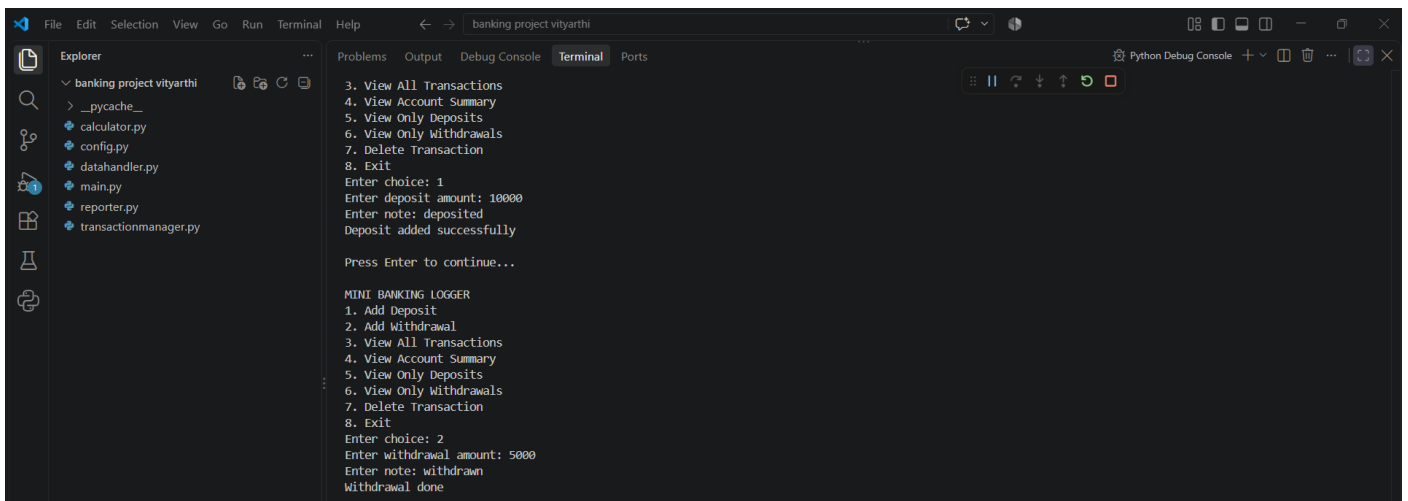


The screenshot shows the Visual Studio Code interface with a Python project named 'banking project vityarthi'. The Explorer pane on the left lists files: __pycache__, calculator.py, config.py, datahandler.py, main.py, reporter.py, and transactionmanager.py. The Terminal pane on the right displays the output of the program. It starts with a 'MINI BANKING LOGGER' and a menu with options 1 through 8. Option 1, 'Add Deposit', is selected. The user is prompted to enter a deposit amount (10000) and a note ('deposited'). The program confirms 'Deposit added successfully' and prompts the user to press Enter to continue.

```
MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 1
Enter deposit amount: 10000
Enter note: deposited
Deposit added successfully

Press Enter to continue...
```

2.Add Withdrawal



This screenshot shows the continuation of the program's execution. The menu is displayed again, and option 2, 'Add Withdrawal', is selected. The user enters a withdrawal amount of 5000 and the note 'withdrawn'. The program outputs 'Withdrawal done' and prompts the user to press Enter to continue. Below this, the menu is shown again with option 3, 'View All Transactions', selected.

```
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 1
Enter deposit amount: 10000
Enter note: deposited
Deposit added successfully

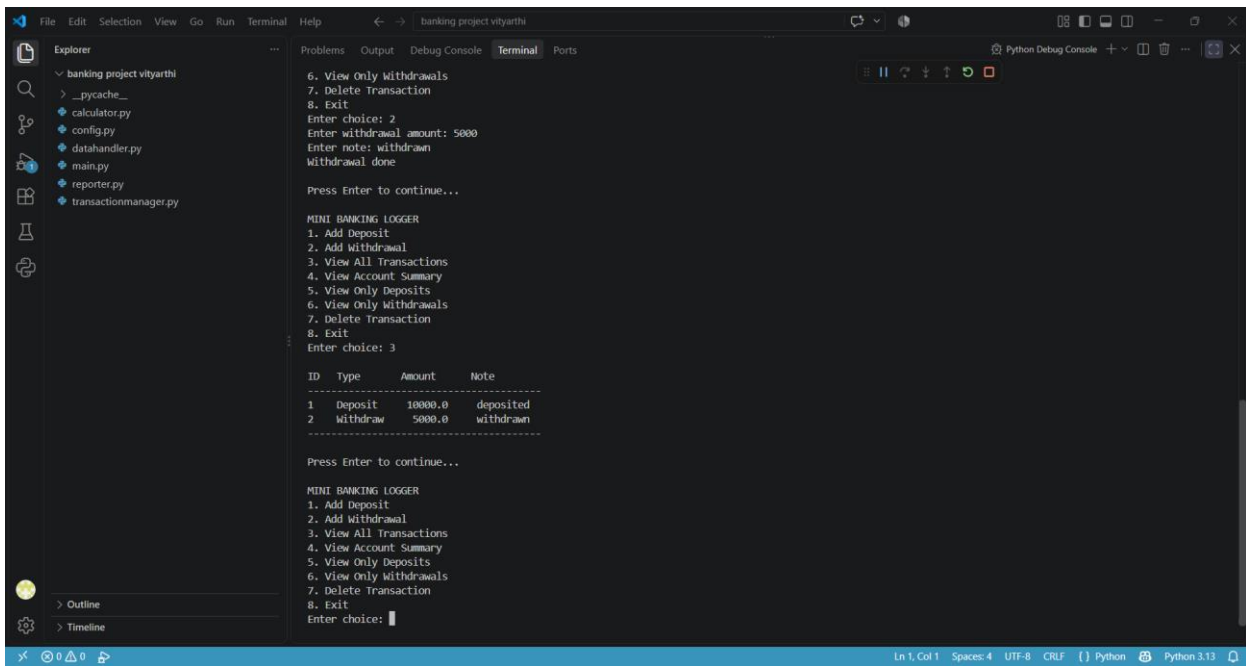
Press Enter to continue...

MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 2
Enter withdrawal amount: 5000
Enter note: withdrawn
Withdrawal done

Press Enter to continue...

MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 3
```

3.View All Transactions



```
File Edit Selection View Go Run Terminal Help
banking project vityarthi

Explorer
banking project vityarthi
  _pycache_
  calculator.py
  config.py
  datahandler.py
  main.py
  reporter.py
  transactionmanager.py

Problems Output Debug Console Terminal Ports
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 2
Enter withdrawal amount: 5000
Enter note: withdraw
Withdrawal done

Press Enter to continue...

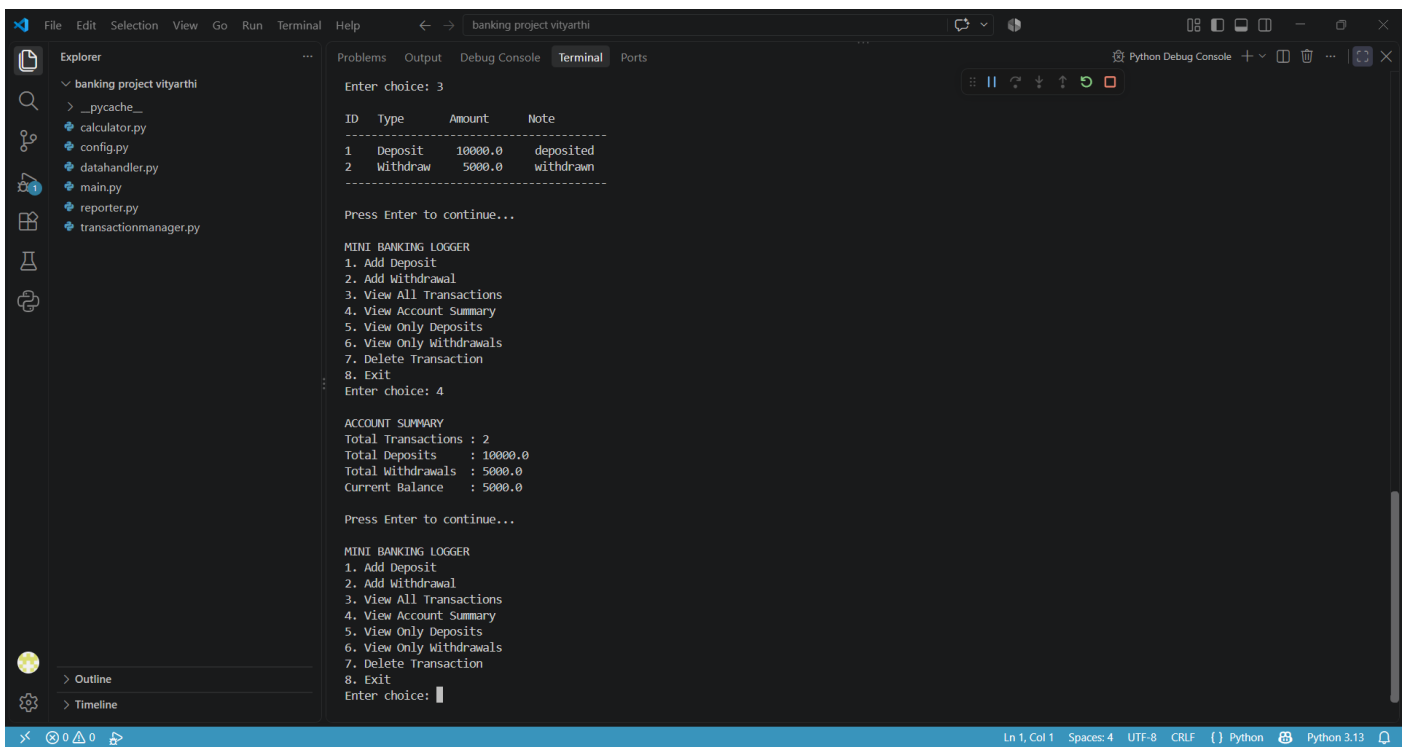
MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 3

ID Type Amount Note
-----
1 Deposit 10000.0 deposited
2 Withdraw 5000.0 withdrawn

Press Enter to continue...

MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 
```

4.View Account Summary



```
File Edit Selection View Go Run Terminal Help
banking project vityarthi

Explorer
banking project vityarthi
  _pycache_
  calculator.py
  config.py
  datahandler.py
  main.py
  reporter.py
  transactionmanager.py

Problems Output Debug Console Terminal Ports
Enter choice: 3

ID Type Amount Note
-----
1 Deposit 10000.0 deposited
2 Withdraw 5000.0 withdrawn

Press Enter to continue...

MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 4

ACCOUNT SUMMARY
Total Transactions : 2
Total Deposits : 10000.0
Total Withdrawals : 5000.0
Current Balance : 5000.0

Press Enter to continue...

MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 
```

5.View Only Deposits

```
8. Exit
Enter choice: 4

ACCOUNT SUMMARY
Total Transactions : 2
Total Deposits    : 10000.0
Total Withdrawals : 5000.0
Current Balance   : 5000.0

Press Enter to continue...

MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 5

ID  Type      Amount    Note
-----
1   Deposit   10000.0   deposited

Press Enter to continue...

MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 
```

6.View Only Withdrawals

```
7. Delete Transaction
8. Exit
Enter choice: 5

ID  Type      Amount    Note
-----
1   Deposit   10000.0   deposited

Press Enter to continue...

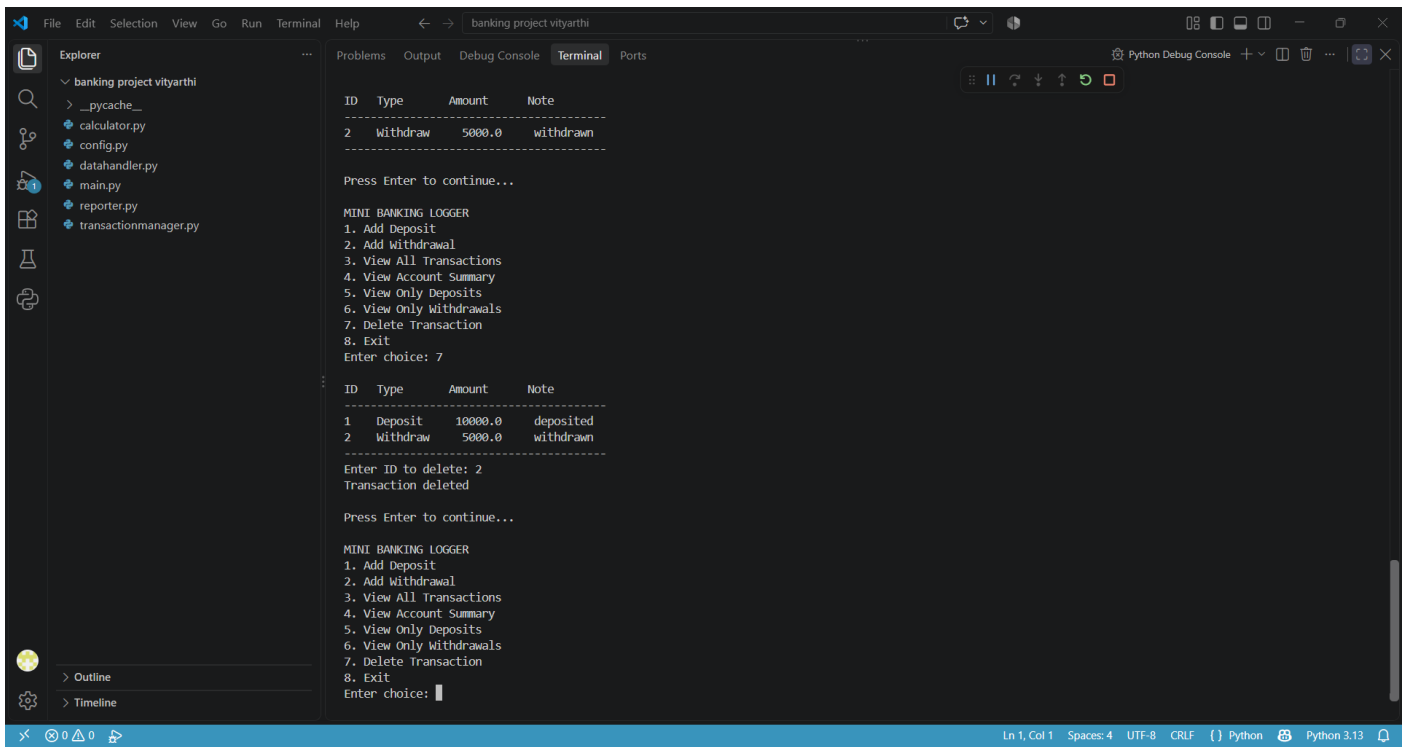
MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 6

ID  Type      Amount    Note
-----
2   Withdraw  5000.0    withdrawn

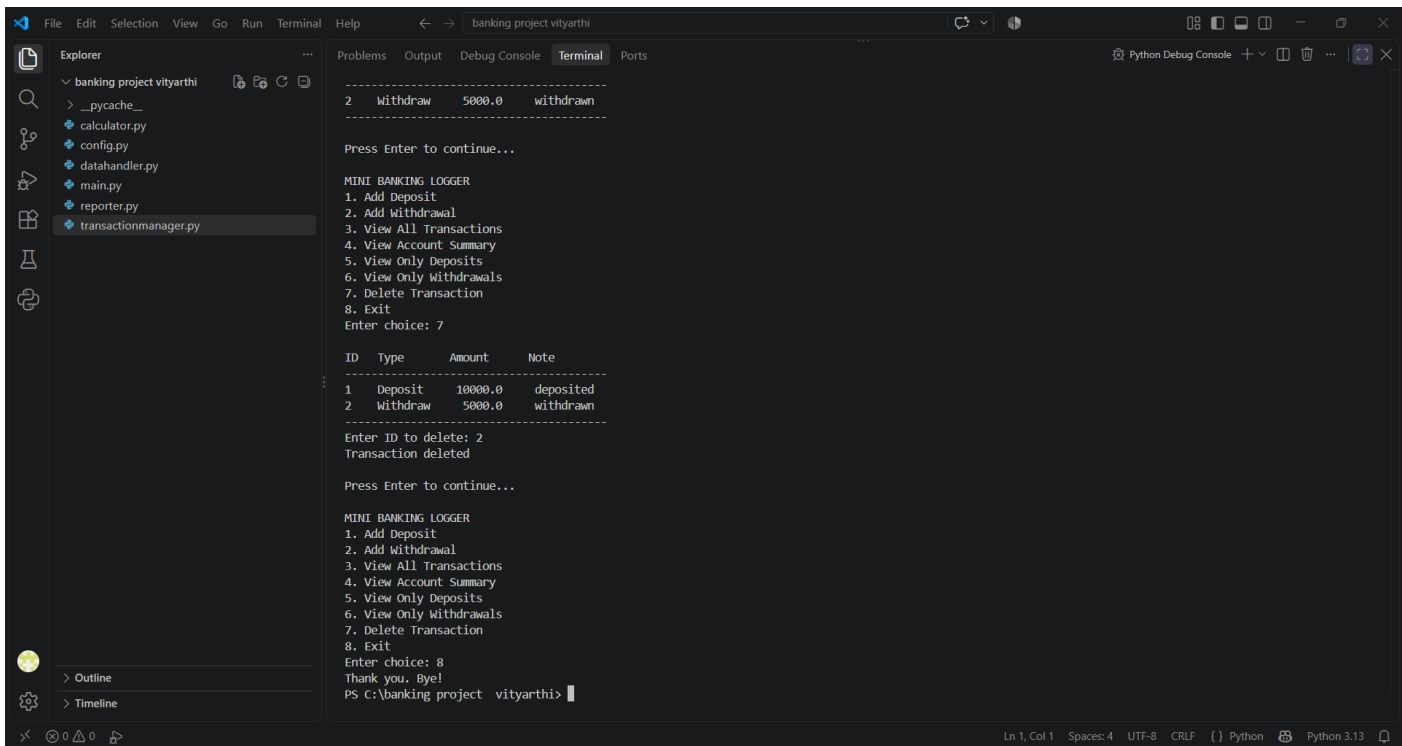
Press Enter to continue...

MINI BANKING LOGGER
1. Add Deposit
2. Add Withdrawal
3. View All Transactions
4. View Account Summary
5. View Only Deposits
6. View Only Withdrawals
7. Delete Transaction
8. Exit
Enter choice: 
```

7.Delete Transaction



8.Exit



Challenges Faced:

One small challenge was handling the case when the user tries to withdraw more money than the available balance. This was solved by checking the current balance before allowing the withdrawal. Another small issue was making sure the transaction ID is unique. This was solved by finding the highest existing ID and adding one to it.

Learnings and Key Takeaways:

Through this project I learned how to break a problem into smaller parts and write separate functions for each part. I also got better practice in using lists and dictionaries. Working with a menu-driven program helped me understand how real programs take input from the user and respond accordingly. Overall it improved my confidence in writing modular Python code.

Future Enhancements:

- Add a feature to search transaction by note
- Allow editing of an existing transaction
- Show transactions in sorted order by amount
- Add a simple date for each transaction

References:

1. Course materials of CSE1021 – Introduction to Problem Solving and Programming
2. Python official documentation (basic data structures)
3. Class notes on lists, dictionaries, functions and control flow
4. VITyarthi course (Python Essentials by Dr. Laxmi D.)
5. Python Programming using problem solving approach (By Reema Thareja)

The application allows the user to enter their weight and height and calculates BMI using the standard BMI formula. Based on the calculated BMI, the application displays a general BMI category and neutral feedback.

The system also provides general balanced food suggestions, a healthy daily routine, and educational information about BMI and its limitations.

PROBLEM STATEMENT:

Many people want a simple way to calculate BMI and understand basic information related to healthy routines. However, beginners may find health-related information difficult to understand.

The purpose of this project is to develop a simple menu-driven Python application that calculates BMI and provides general educational information about balanced food choices, daily routines, and BMI.

The application is designed for educational purposes and does not provide medical diagnosis or personalized medical advice.

The information use in this project is based on WHO (World Health Organization) Released data related to BMI.

OBJECTIVE:

->The main objectives of the project are:

- To calculate BMI using the user's weight and height.
- To display a general BMI category based on the calculated value.
- To provide general balanced food suggestions.
- To provide general healthy daily routine guidance.
- To explain BMI and its limitations.
- To demonstrate modular programming using Python.
- To implement input validation and basic error handling.
- To test the application using different valid and invalid inputs.

Functional Requirements:

- BMI Calculation

Input: Weight in kilograms, Height in centimetres

Processing: The program converts height from centimetres to metres and calculates BMI.

Output: BMI value, General BMI category, Neutral feedback.

- **Food Suggestions:** The system provides general balanced meal suggestions based on the BMI category obtained from the BMI calculation.
- **Healthy Daily Routine** The system provides general suggestions for: Morning, Daytime, Evening, Night .
- **Learn About BMI** The system explains: Meaning of BMI, BMI formula Purpose of BMI, Limitations of BMI.

- **Main Menu and Program Control**

The main program provides a menu through which the user can select

different features and exit the application.

NON-FUNCTIONAL REQUIREMENTS:

- **Usability**
The application uses a simple menu-driven interface so that users can select features using numbered options.
- **Reliability**
The program handles invalid numerical inputs and non-positive values without terminating unexpectedly.
- **Maintainability**
The application is divided into separate Python modules. Each module is responsible for a specific functionality, making the code easier to understand and modify.
- **Error Handling**

The application uses input validation and exception handling to handle invalid user input.

- Performance

BMI calculations and information display are performed quickly because the application uses simple calculations and does not require external services or a database.

SYSTEM ARCHITECTURE

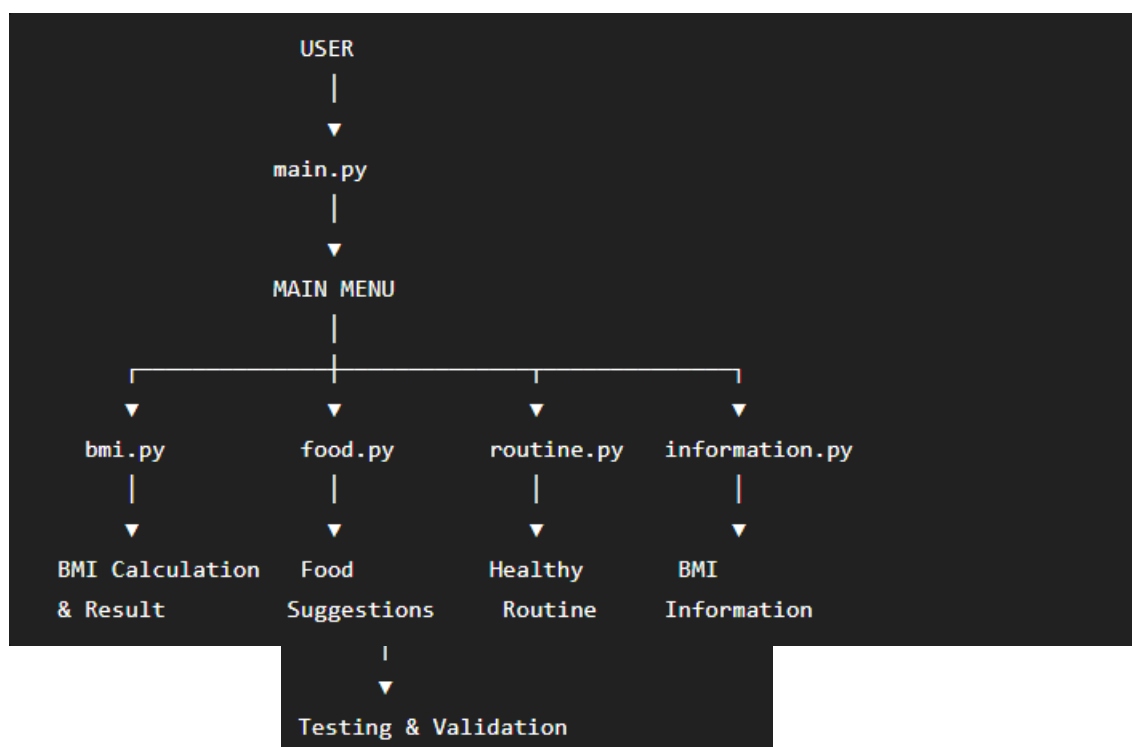
The application follows a simple modular architecture.

The main.py module acts as the main controller and displays the menu. Based on the user's selection, it calls the appropriate functional module.

Components

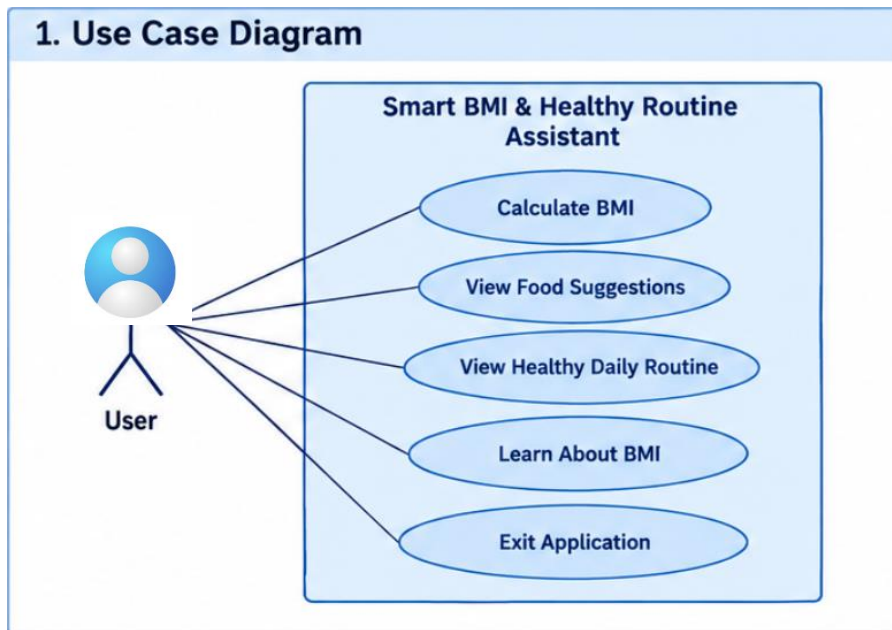
- main.py — Main menu and program control
- bmi.py — BMI calculation and input validation
- food.py — Food suggestions
- routine.py — Healthy daily routine
- information.py — BMI educational information
- Testing.ipynb — Testing and validation

SYSTEM ARCHITECTURE

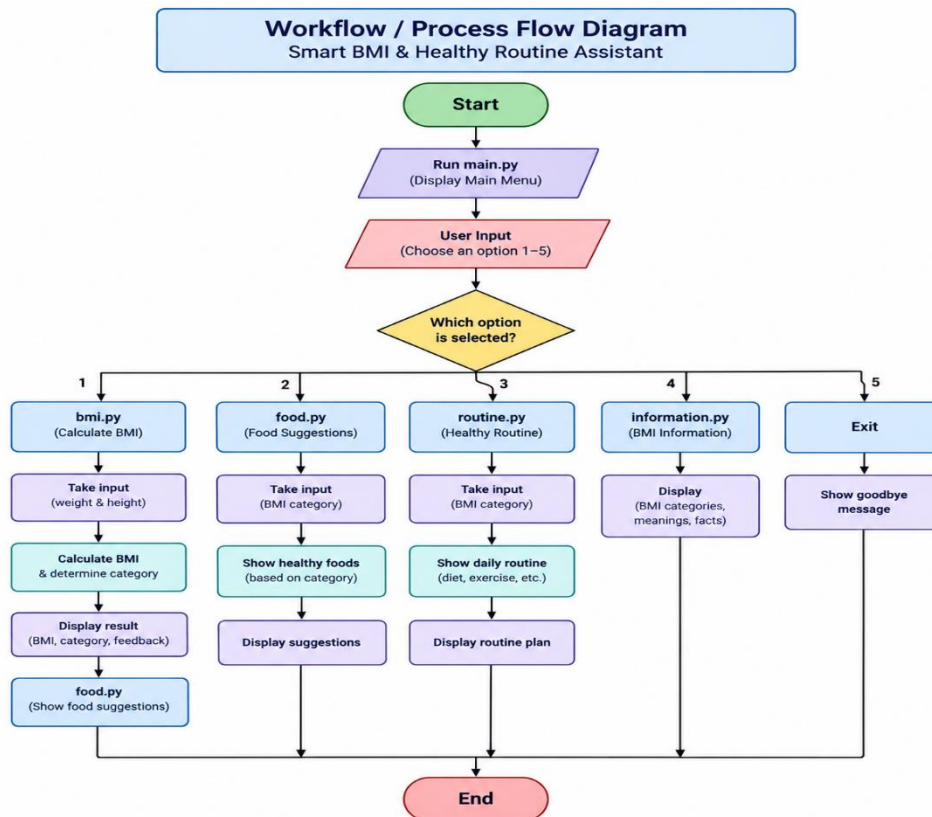


DESIGN DIAGRAMS

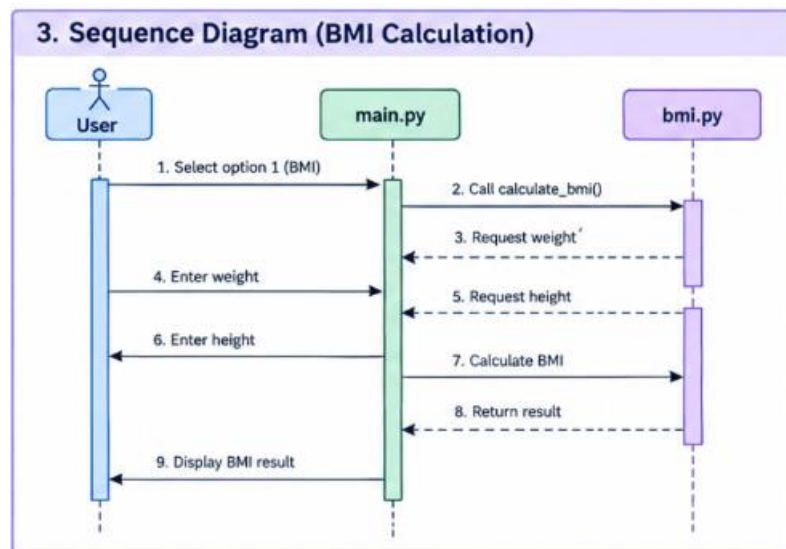
1. Use Case Diagram



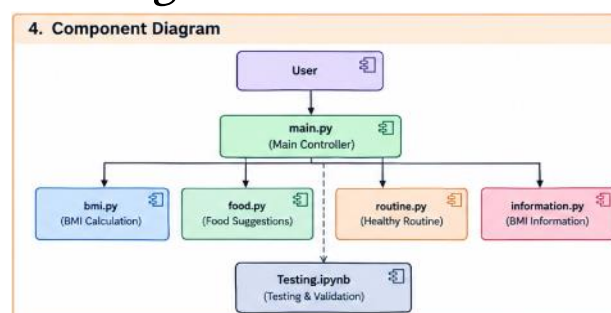
2. Workflow Diagram



3. Sequence Diagram



4. Class/Component Diagram



Design Decisions & Rationale

- Modular Structure

The project is divided into multiple Python files so that each major function has its own module. This improves organization and maintainability.

- Menu-Driven Interface

A numbered menu was selected because it provides a simple interaction method for a beginner-friendly command-line application.

- Input Validation

Input validation was included to prevent invalid values from causing program errors.

- No Database

The current application does not store user information or history. Therefore, a database is not required.

- Command-Line Execution

The application can be executed directly through the terminal using:

```
python main.py
```

This allows the project to run without requiring a graphical interface.

IMPLEMENTATION DETAILS

The project is implemented in Python using a modular structure.

main.py

Controls the main menu and connects the different modules.

bmi.py

Accepts weight and height, validates the input, calculates BMI, and displays the category and feedback.

food.py

Displays general balanced food suggestions based on the BMI category returned by the BMI module.

routine.py

Displays general healthy daily routine information.

information.py

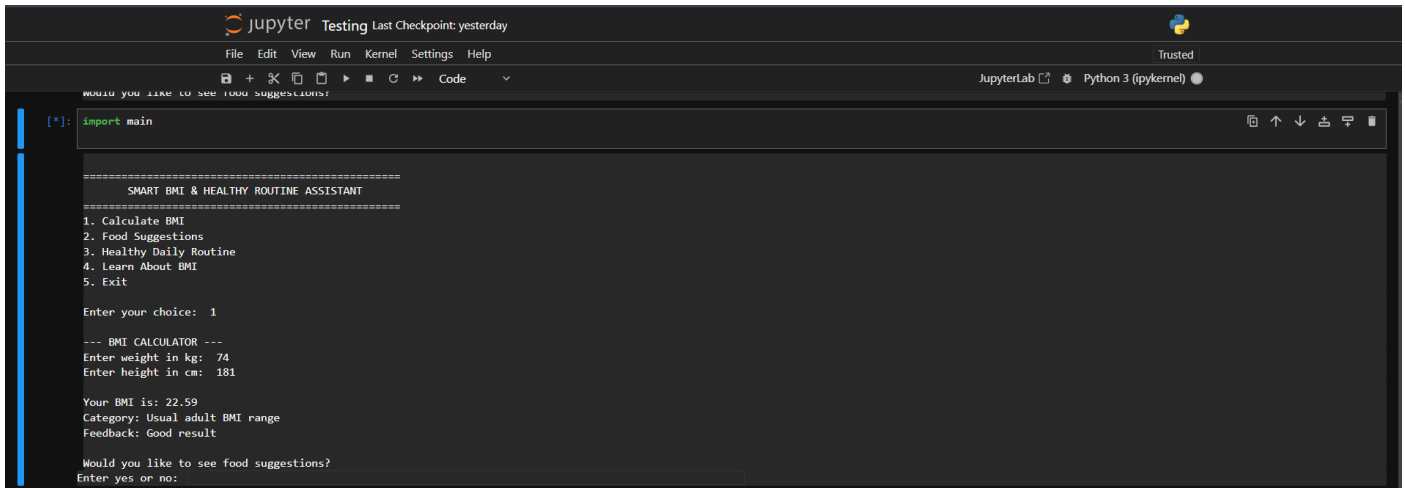
Provides educational information about BMI, its formula, and its limitations.

Testing.ipynb

Used to test the application with different inputs and verify the expected outputs.

SCREENSHOTS / RESULTS

Screenshot 1 — BMI Calculation



```
[*]: import main

=====
SMART BMI & HEALTHY ROUTINE ASSISTANT
=====
1. Calculate BMI
2. Food Suggestions
3. Healthy Daily Routine
4. Learn About BMI
5. Exit

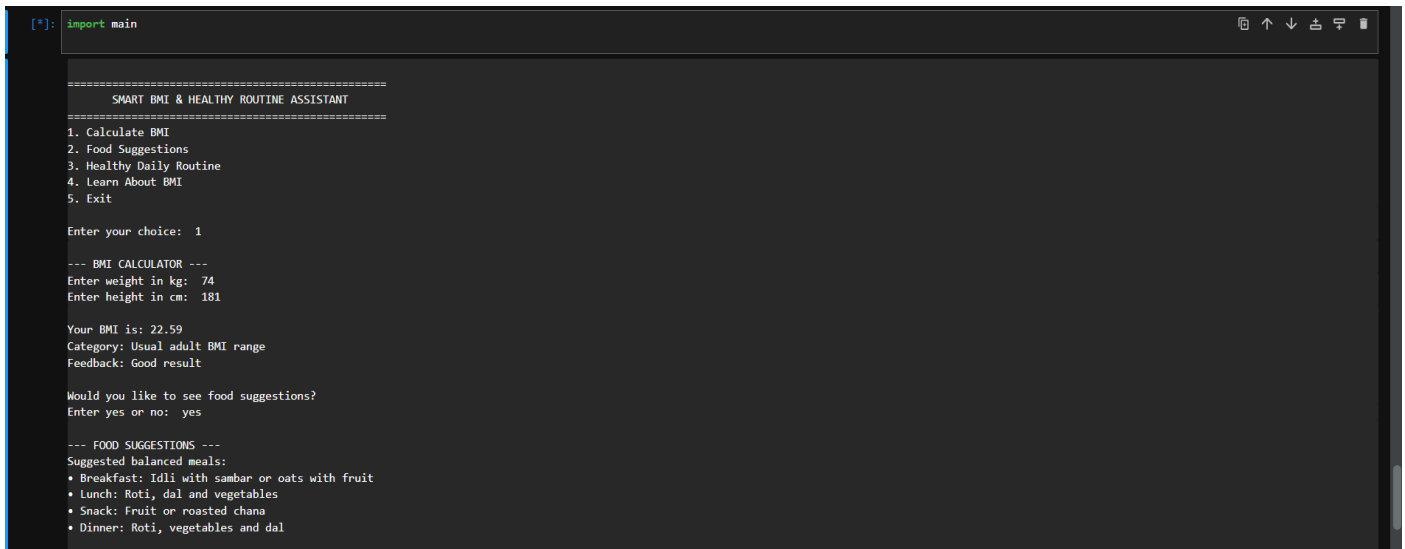
Enter your choice: 1

--- BMI CALCULATOR ---
Enter weight in kg: 74
Enter height in cm: 181

Your BMI is: 22.59
Category: Usual adult BMI range
Feedback: Good result

Would you like to see food suggestions?
Enter yes or no:
```

Screenshot 2 — Food Suggestions



```
[*]: import main

=====
SMART BMI & HEALTHY ROUTINE ASSISTANT
=====
1. Calculate BMI
2. Food Suggestions
3. Healthy Daily Routine
4. Learn About BMI
5. Exit

Enter your choice: 1

--- BMI CALCULATOR ---
Enter weight in kg: 74
Enter height in cm: 181

Your BMI is: 22.59
Category: Usual adult BMI range
Feedback: Good result

Would you like to see food suggestions?
Enter yes or no: yes

--- FOOD SUGGESTIONS ---
Suggested balanced meals:
• Breakfast: Idli with sambar or oats with fruit
• Lunch: Roti, dal and vegetables
• Snack: Fruit or roasted chana
• Dinner: Roti, vegetables and dal
```

Screenshot 3 — Healthy Daily Routine



```
[*]: import main

=====
SMART BMI & HEALTHY ROUTINE ASSISTANT
=====
1. Calculate BMI
2. Food Suggestions
3. Healthy Daily Routine
4. Learn About BMI
5. Exit

Enter your choice: 3

--- HEALTHY DAILY ROUTINE ---

Morning:
• Start your day at a regular time.
• Drink water regularly.
• Have a balanced breakfast.

Daytime:
• Take regular movement breaks.
• Include fruits and vegetables in meals.

Evening:
• Include some age-appropriate physical activity.
• Have a balanced meal.

Night:
• Follow a regular sleep schedule.
• Reduce unnecessary screen use before sleeping.
```

Screenshot 4 — Learn About BMI

```
[*]: import main

Enter your choice: 4

--- ABOUT BMI ---

BMI stands for Body Mass Index.

BMI is calculated using:

BMI = Weight / Height2

BMI uses height and weight as a general screening measure.

It does not give a complete picture of health.

The food and routine suggestions in this project are general educational information.
```

Screenshot 5 — Error Handling

```
Er (C:\Users\Arun\OneDrive\Desktop\project\Smart BMI Assistant\main.py)

[ ]: import main

=====
SMART BMI & HEALTHY ROUTINE ASSISTANT
=====
1. Calculate BMI
2. Food Suggestions
3. Healthy Daily Routine
4. Learn About BMI
5. Exit

Enter your choice: 1

--- BMI CALCULATOR ---
Enter weight in kg: abc

Invalid input. Please enter numbers only.

Would you like to see food suggestions?
```

12. TESTING APPROACH

The application was tested using both valid and invalid inputs.

Test	Input	Expected Result
Good BMI	74Kg, 181cm	BMI calculated correctly
Invalid weight	abc	Error message displayed
Health Daily routine	3	Correct output
Learn About BMI	4	Correct output
EXIT	5	Correct output

CHALLENGES FACED

During this, several challenges were encountered:

1. Understanding how to divide the program into separate Python modules.
2. Connecting different modules using Python imports.
3. Handling invalid user input without terminating the program.
4. Testing modules and functions using Jupyter Notebook.
5. Organizing the project files into a proper folder structure.
6. Setting up and uploading the project to GitHub.
7. Preparing the project documentation and diagrams.

These challenges helped improve understanding of Python programming, modular design, testing, and version control.

LEARNINGS & KEY TAKEAWAYS

Through this project, the following concepts were learned:

- Python functions and conditional statements
- Python modules and imports
- Input validation
- Exception handling
- Modular project organization
- Testing and debugging
- GitHub repository management
- Project documentation
- Creating system design diagrams
- Learned Project uploading in GitHub

FUTURE ENHANCEMENTS

Possible future improvements include:

1. Developing a graphical user interface.
2. Adding more detailed educational health information.
3. Adding optional data storage for user records.
4. Adding more comprehensive testing.
5. Improving the user interaction and presentation.
6. Adding additional educational health-related features.
7. Improving the project's documentation and visualization.
8. Converting this project into a website using applications like streamlit.

REFERENCES

1. Vidyarthi – Build Your Own Project, General Project Instructions & Submission Guidelines.
2. Python Documentation, Python programming language documentation.
3. Python BY EXAMPLE By-Nichola Lacey

